WARWICK

SCHOOL OF ENGINEERING

Lab 3 – Running HLS code on an FPGA

Module Code:

ES4F3/ES3F1

Module Title:

High Performance Embedded Systems Design

Last Update: Jan 2025

Lab 3

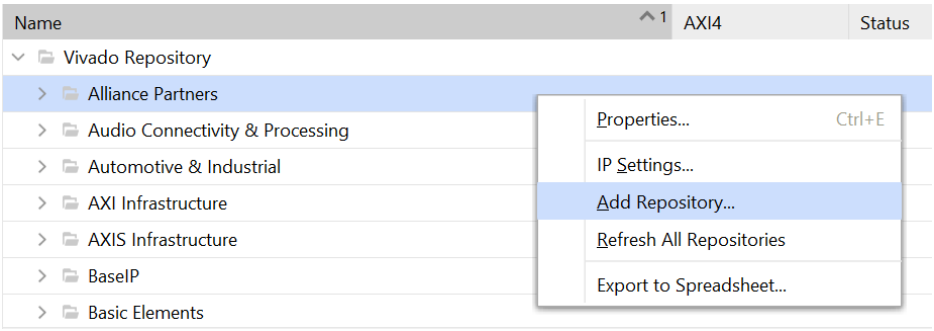
Lab 3 builds on the previous lab, integrating your HLS design for matrix multiplication within a SoC. Vivado will be used in this lab to interconnect the HLS block with the Zynq hard processor. Vitis will be used to run the C code and take measurements on the execution time in a similar way to lab 1.

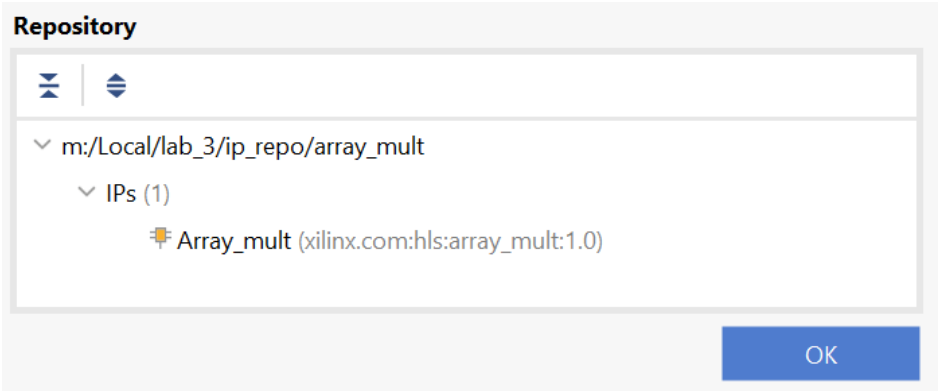
Vitis HLS Setup

- You can use the project from last lab for this.
- Place pragmas to fully unroll the two inner for-loops (COLS LOOP and MULT ACC LOOP).
- Make sure you have Synthesised and Packaged the HLS component.
- Copy the contents of the zip file created with the same name as the root function of HLS component to a new folder named “ip_repo/array_mult/” in this labs folder.
- This zip folder is found in the “hls_component/<component_name>/” or “<lab name>/<component_name>/” folder of the Vitis workspace.

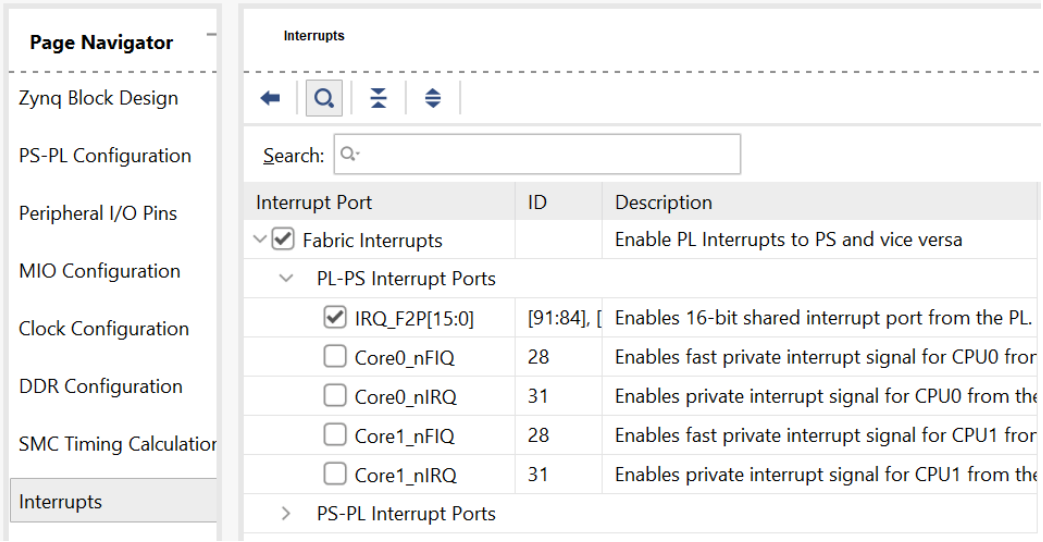
Vivado Setup

- Create a Vivado project as in previous labs.
- Add the IP repository of your HLS design by clicking on the “Window” menu and selecting “IP Catalog”. Right click on the new window that opens and click on “Add Repository”. Browse to the directory where you extracted your HLS IP.

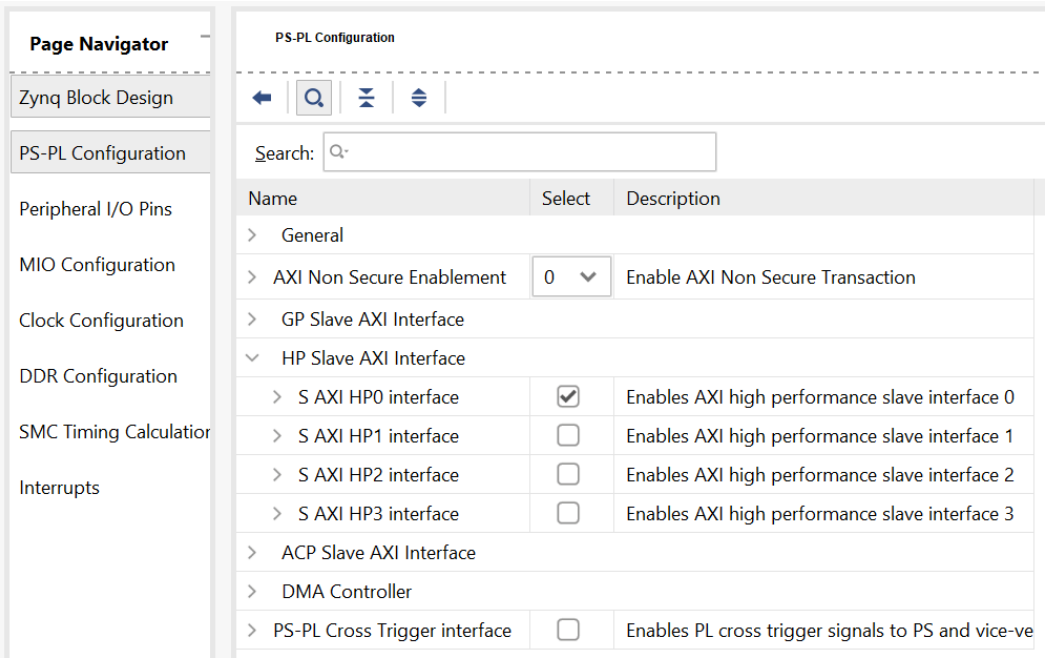




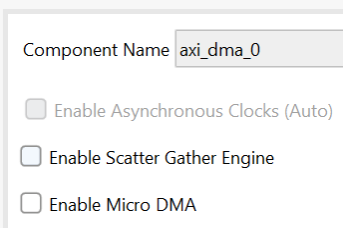
- Create and open a new block design.
- By right clicking in the diagram's area add the following IPs:
    - ZYNQ7 Processing System
    - Processor System Reset
    - Concat
    - AXI Direct Memory Access
    - AXI4-Stream Data FIFO (×2)
    - The HLS IP core for matrix multiplication
- Double click on the "ZYNQ7 Processing System" block. On the window that opens up, select the "Interrupts" tab on the left-hand side. Enable and then expand the "Fabric Interrupts" and then, from the "PL-PS Interrupt Ports", enable the "IRQ F2P[15:0]". (Don't click OK yet, more configurations follow)



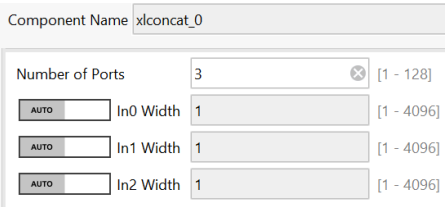
- On the same window, click on the "PS-PL configuration" tab, expand the "HP Slave AXI Interface" and enable the "S AXI HP0 interface". Then click OK.



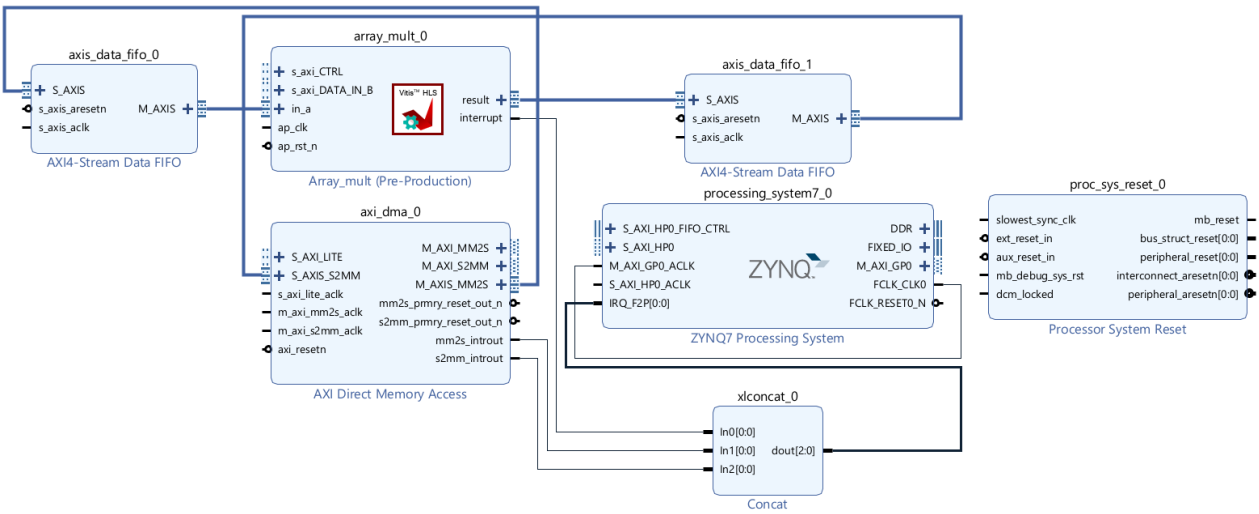
- Double click on the “AXI Direct Memory Access” block. On the window that opens, unselect the Enable Scatter Gather Engine, and then click OK.



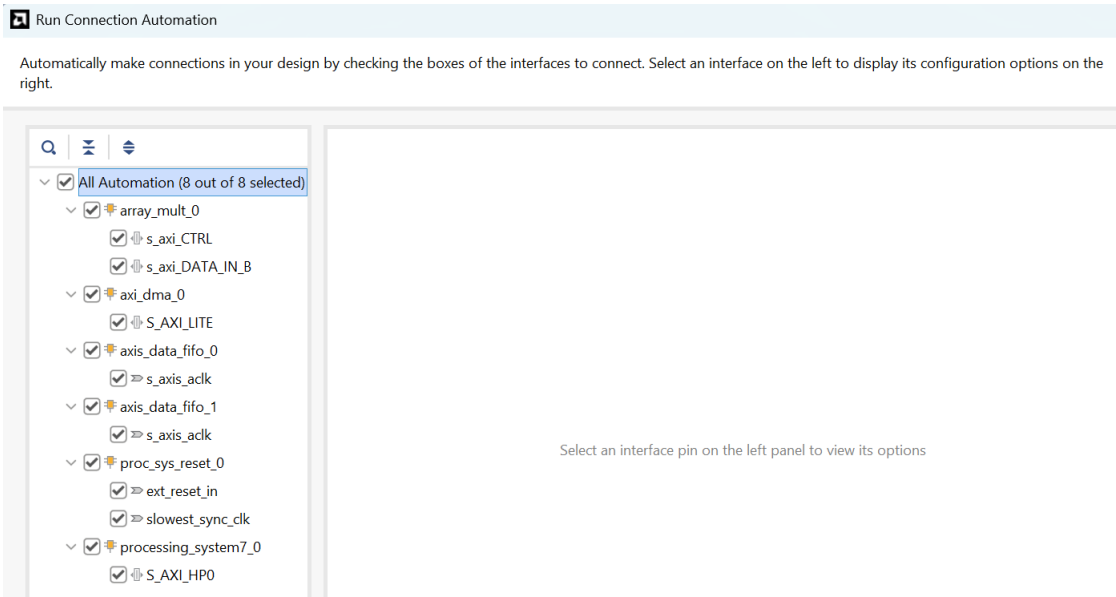
- Double click on the Concat. On the Number of Ports box, write 3, and then click OK.



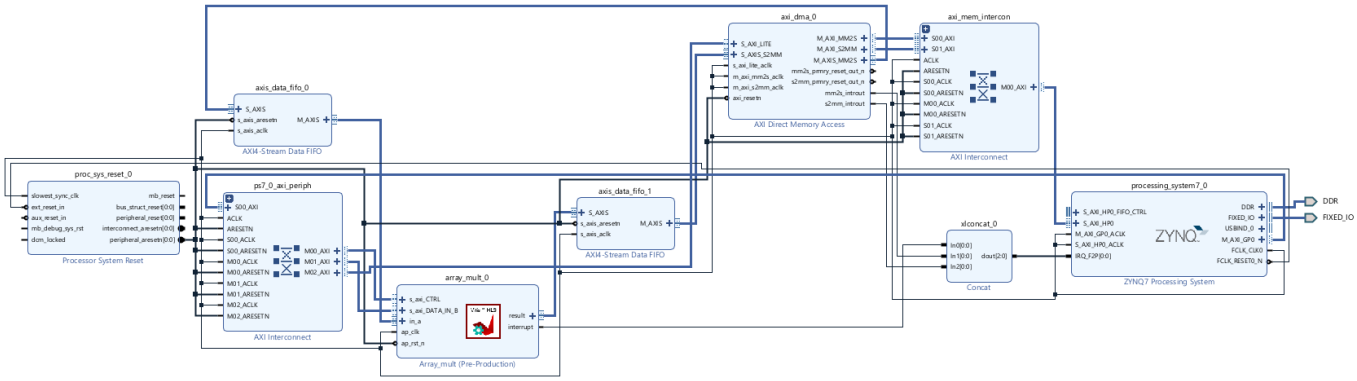
- Make the following connections:
  - The “FCLK_CLK0” of the “ZYNQ7 Processing System” to the “M_AXI_GP0_ACLK” on the same block.
  - The “dout” of the “Concat” block to the “IRQ_F2P” of the ZYNQ7 Processing System.
  - The “interrupt” output of your HLS block to the “In0” input on the “Concat” block.
  - Connect “mm2s_introut” and “s2mm_introut” outputs of the “AXI Direct Memory Access” block to “In1” and “In2” on the “Concat” block respectively.
  - The “M_AXIS” output of the first FIFO to the “in_a” of your HLS IP.
  - The output “result” of your HLS IP to the “S_AXIS” of the second FIFO.
  - The “S_AXIS” of the first FIFO to the “M_AXIS_MM2S” of the “AXI Direct Memory Access” block.
  - The “M_AXIS” of the second FIFO to the “S_AXIS_S2MM” of the “AXI Direct Memory Access” block.



- Click on “Run Block Automation” on the top of your block diagram window. Make sure to select all the IP blocks on the left-hand side and then click OK. Do the same with “Run Connection Automation”.



- If “Run Connection Automation” is still on the top of your block diagram window, run it again. Then regenerate the layout.



- Validate and save the block diagram.
  [You can ignore errors about `PCW_UIPARAM_DDR_DQS_TO_CLK_DELAY` having negative values]
- Go to Sources window, then right click on your design name under Design Sources and then click Create HDL wrapper
- Generate the bitstream

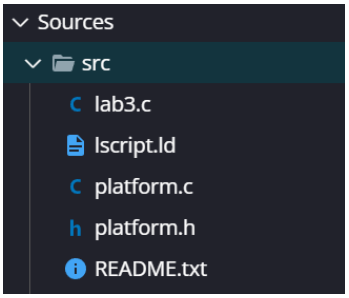
Tcl Console | Messages | Log | Reports | **Design Runs**

Name	Constraints	Status	LUT	FF	BRAM	URAM	DSP
synth_1 (active)	**constrs_1**	**synth_design Complete!**	**0**	**0**	**0**	**0**	**0**
impl_1	constrs_1	write_bitstream Complete!	3834	530	5	0	15
Out-of-Context Module Runs							
design_1		Submodule Runs Complete					
design_1_xbar_0_synth_1	design_1_xbar_0	synth_design Complete!	142	131	0	0	0
design_1_array_mult_0_0_synth_1	design_1_array_mult_0_0	synth_design Complete!	1028	157	1	0	15
design_1_axis_data_fifo_1_0_synth_1	design_1_axis_data_fifo_1_0	synth_design Complete!	49	53	1	0	0
design_1_auto_us_0_synth_1	design_1_auto_us_0	synth_design Complete!	208	322	0	0	0
design_1_auto_pc_0_synth_1	design_1_auto_pc_0	synth_design Complete!	421	562	0	0	0
design_1_proc_sys_reset_0_0_synth_1	design_1_proc_sys_reset_0_0	synth_design Complete!	19	40	0	0	0
design_1_auto_us_1_synth_1	design_1_auto_us_1	synth_design Complete!	243	265	0	0	0
design_1_auto_pc_1_synth_1	design_1_auto_pc_1	synth_design Complete!	398	430	0	0	0
design_1_processing_system7_0_0_synth_1	design_1_processing_system7_0_0	synth_design Complete!	24	0	0	0	0
design_1_xbar_1_synth_1	design_1_xbar_1	synth_design Complete!	286	353	0	0	0
design_1_axis_data_fifo_0_0_synth_1	design_1_axis_data_fifo_0_0	synth_design Complete!	49	53	1	0	0
design_1_axi_dma_0_0_synth_1	design_1_axi_dma_0_0	synth_design Complete!	1378	185	2	0	0

- After the bitstream has been generated, export the hardware (make sure that you included the bitstream) and launch Vitis.

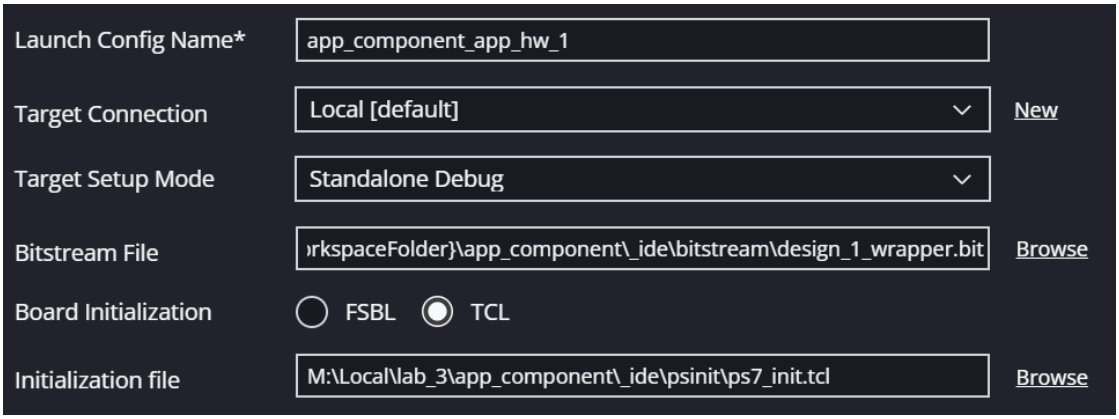
Vitis Application Setup

- Create a platform component as done in lab 1. Use the XSA file generated in the previous step.
- Create an application component as in lab 1. Instead of using the “hello world” example use the “New Component” button to create a blank application.
- Right click on the applications “src/” folder and import files “platform.h”, “platform.c”, and “lab3.c” from this labs folder.



- Build both the platform and the application.
- Run the application with a PuTTY serial connection to view the output as done in lab 1.

  [If you see the error ‘invalid command name "ps7_init"’, open the “launch.json” file in the settings folder of the app component and click “Browse” next to the “Initialization file” and select “vitis_workspace\app_component_ide\psinit\ps7_init.tcl”]



Task

Try implementing your different optimisation designs from lab 2. Compare the different performance and resource utilization on the FPGA.

Extension

If you try to also unroll the third loop in the HLS component, ROWS LOOP, you'll see the DSP utilization exceeds the device's availability. Can you come up with a way to make the design fit to the FPGA while all three loops are fully unrolled?

HINT: Consider the DSP block's (DSP48E1) bit-widths and adjust your variables.